UNIVERSITY OF PISA

Lesson 10

Silvia Giulia Galfrè

4. Normalization

- Differential expression analysis requires that gene expression values should be compared among samples.
- If a gene has more mapped reads, it does not mean that it is differentially expressed
- Normalization is one of the basic steps for differential expression analysis.

scaling factors

- The simplest form of inter-sample normalization: scaling raw read counts in each sample by a single sample-specific factor
- Five common methods for calculating these scaling factors:
  1. Total count (TC): gene counts are divided by the total number of mapped reads (or library size) and multiplied by the mean total count across all the samples of the dataset.
  2. Upper Quartile (UQ): similar in principle to TC, the total counts are replaced by the upper quartile of counts different from 0
  3. Median (Med): similar to TC, the total counts are replaced by the median counts different from 0

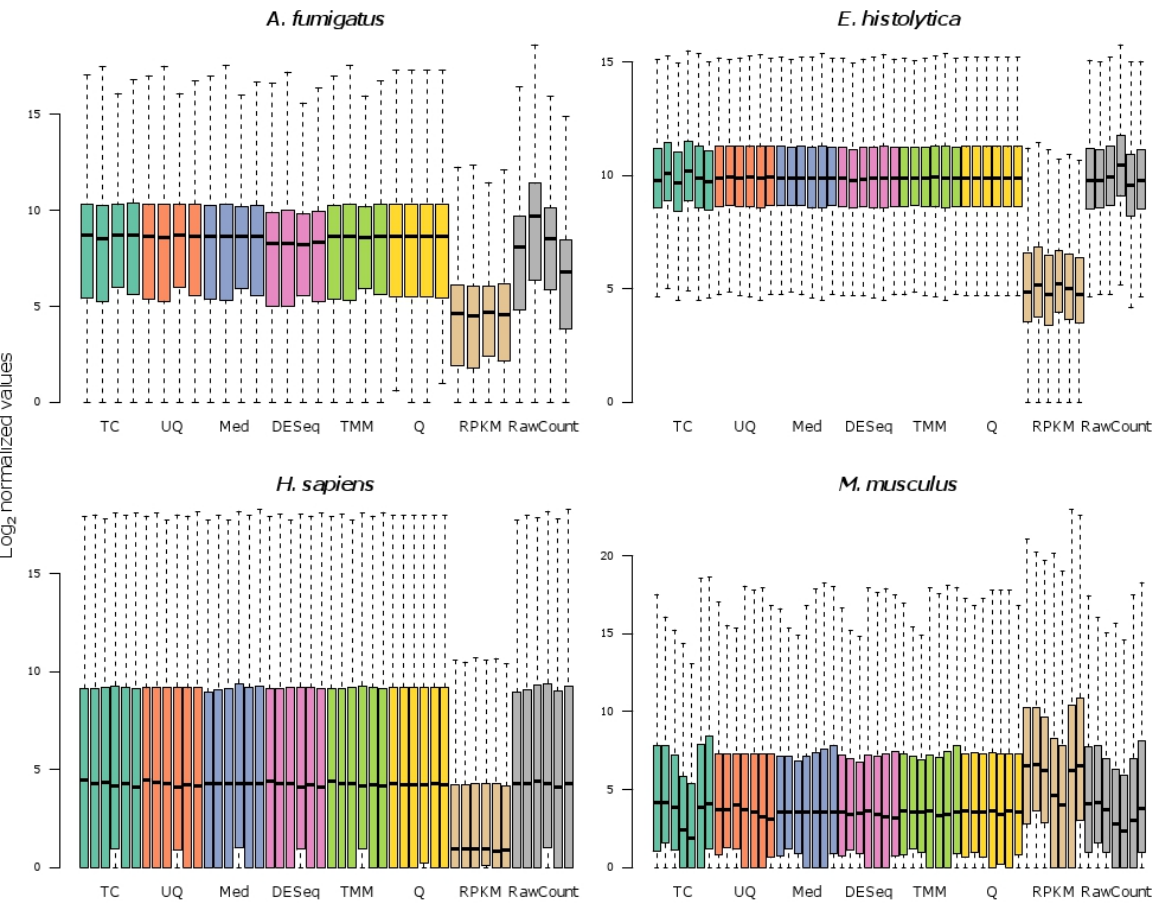
scaling factors

4. DESeq: normalization method included in the DESeq package;
   - based on the hypothesis that most genes are not DE.
   - The scaling factor is computed as the median of the ratio, for each gene, of its read count over its geometric mean across all samples.
   - The underlying idea is that non-DE genes should have similar read counts across samples, leading to a ratio of 1.
   - The median of this ratio provides an estimate of the correction factor that should be applied to all read counts of this sample to fulfill the hypothesis.
5. Trimmed Mean of M-values (TMM): method implemented in the edgeR package;
   - it is based on the hypothesis that most genes are not DE.
   - The factor is computed with one sample being considered as a reference and the others as test samples.
   - For each test sample, TMM is computed as the weighted mean of log ratios between this test and the reference.
   - To obtain normalized read counts, these normalization factors are re-scaled by the mean of the normalized library sizes. Normalized read counts are obtained by dividing raw read counts by these re-scaled normalization factors.

Other two alternative

1. Quantile (Q): First proposed in the context of microarray data, this normalization method consists in matching distributions of gene counts across samples. It is implemented in the Bioconductor package limma.
2. Reads Per Kilobase per Million mapped reads (RPKM): re-scales gene counts to correct for differences in both library sizes and gene length. However, it has been shown that attempting to correct for differences in gene length in a differential analysis actually has the effect of introducing a bias in the per-gene variances, in particular for lowly expressed genes.

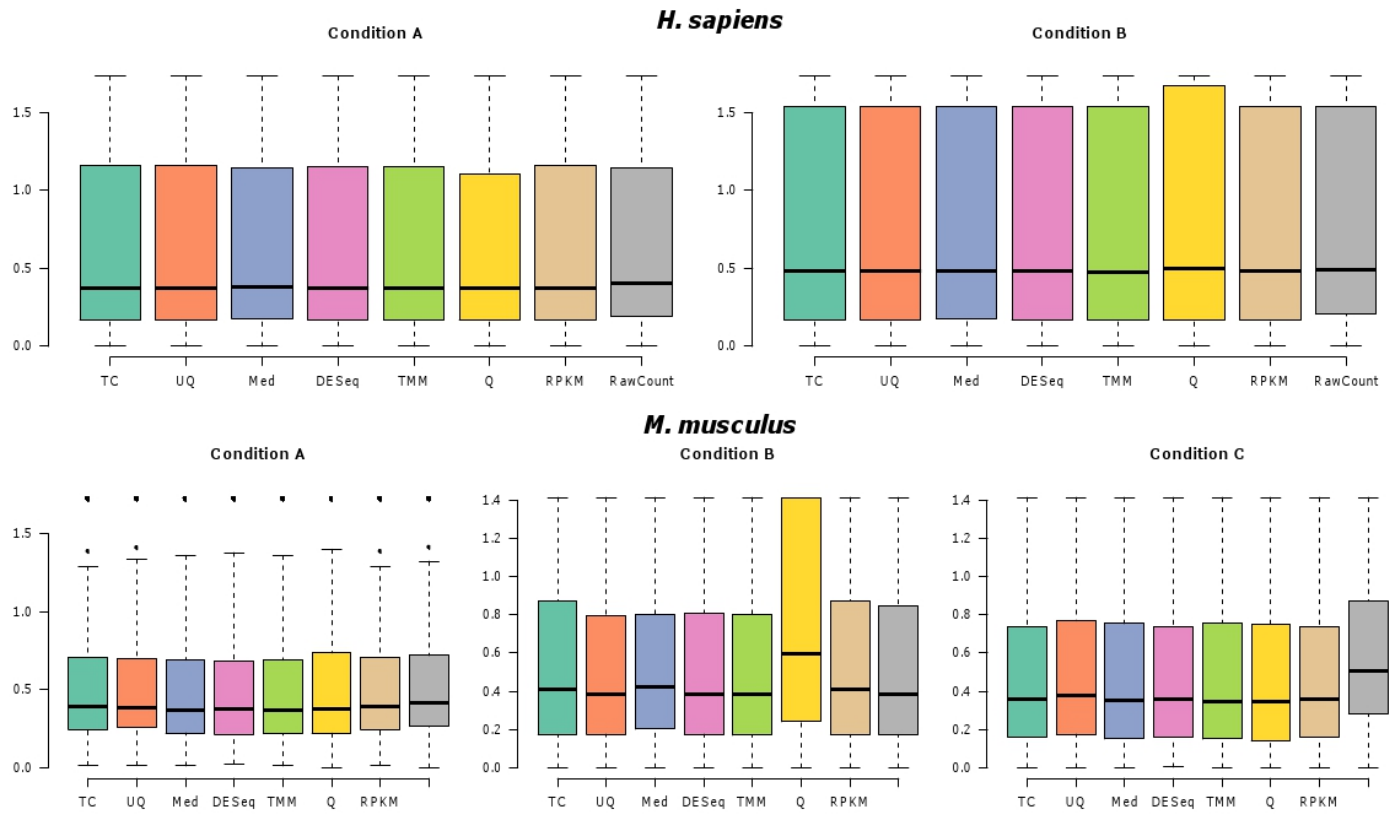
Comparison of normalization methods



Boxplots of log2 for all conditions and replicates by normalization method

Dillies, M.-A. *et al.* A comprehensive evaluation of normalization methods for Illumina high-throughput RNA sequencing data analysis. *Briefings in Bioinformatics* **14**, 671–683 (2013).

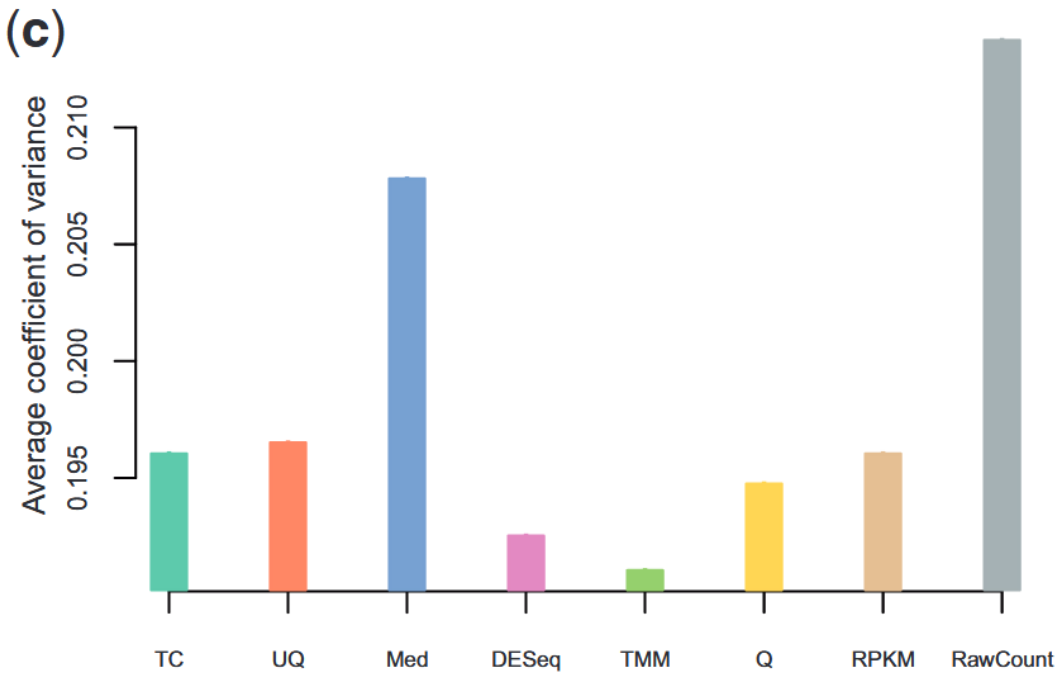
Comparison of normalization methods



Boxplots of coefficient of variation for all conditions and replicates by normalization method

Dillies, M.-A. *et al.* A comprehensive evaluation of normalization methods for Illumina high-throughput RNA sequencing data analysis. *Briefings in Bioinformatics* **14**, 671–683 (2013).

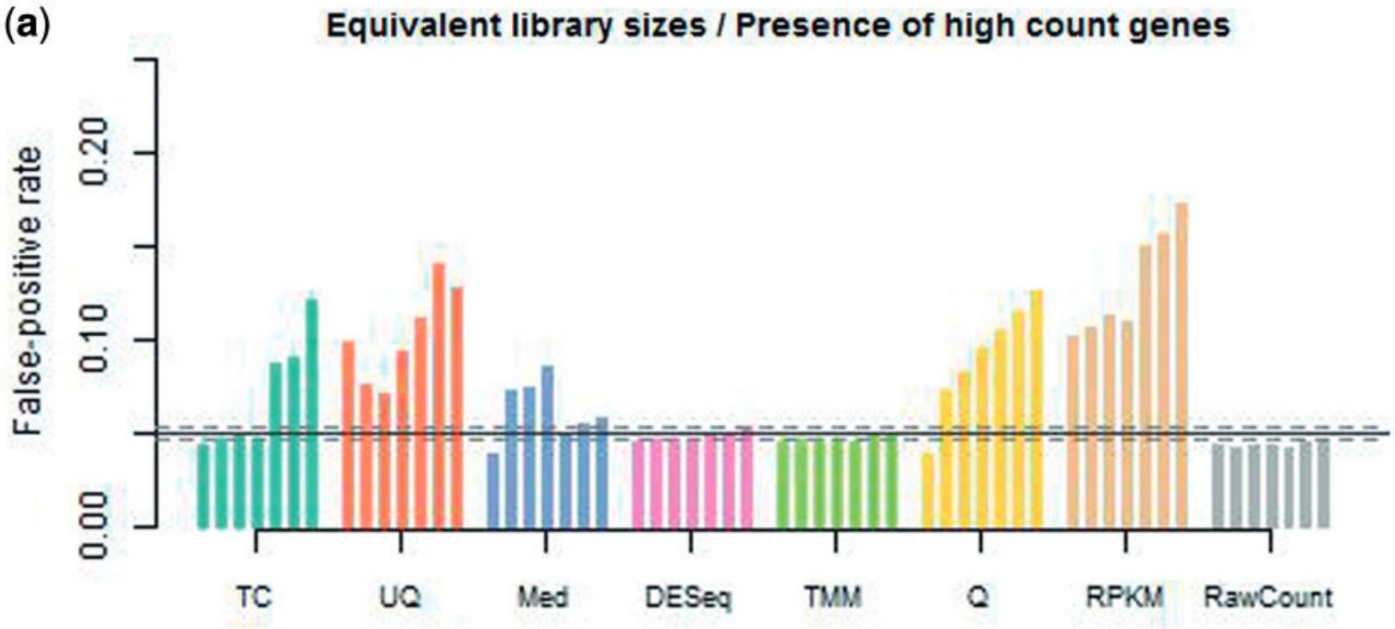
Comparison of normalization methods



Analysis of housekeeping genes for the H. sapiens data.

Dillies, M.-A. *et al.* A comprehensive evaluation of normalization methods for Illumina high-throughput RNA sequencing data analysis. *Briefings in Bioinformatics* **14**, 671–683 (2013).

Comparison of normalization methods

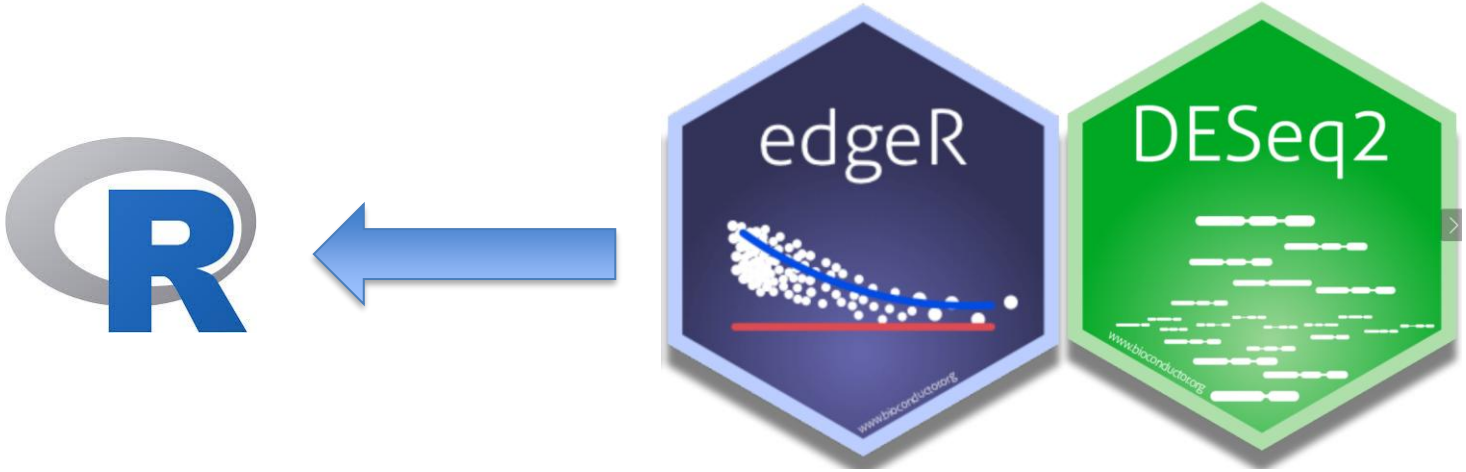


Comparison of normalization methods for simulated data. Average false-positive rate over 10 independent datasets simulated with varying proportions of differentially expressed genes (from 0% to 30% for each normalization method).

Dillies, M.-A. *et al.* A comprehensive evaluation of normalization methods for Illumina high-throughput RNA sequencing data analysis. *Briefings in Bioinformatics* **14**, 671–683 (2013).

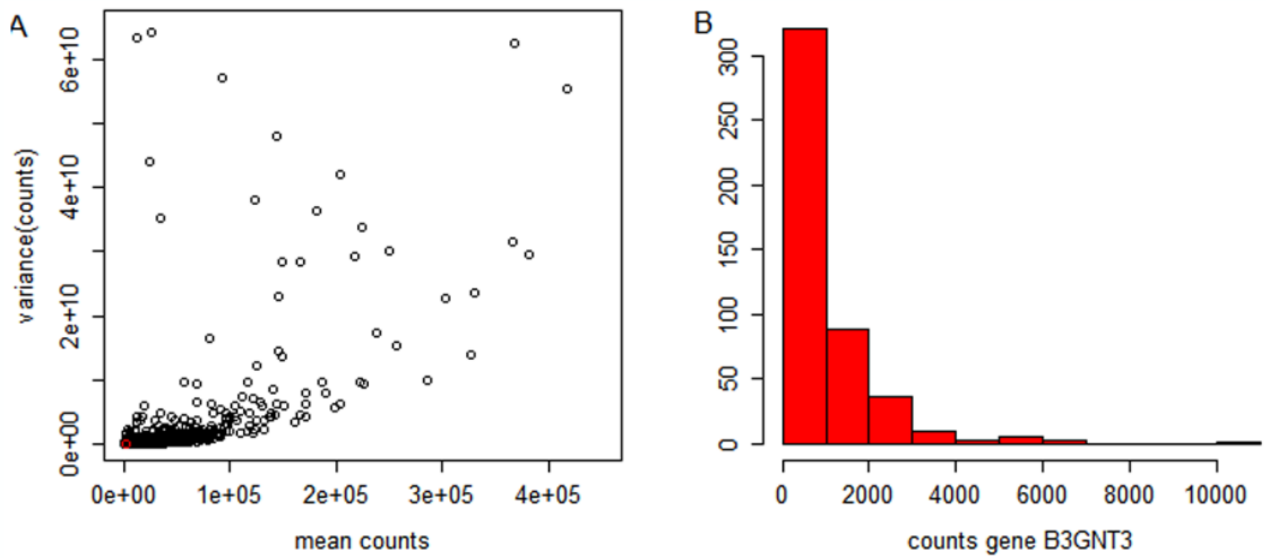
Conclusion

- Two best possible choices are:
  - DESEQ
  - TMM
- Means: DESeq2 and EdgeR



Characteristics of RNA-seq data

- Variance depends on the mean count
- Counts are non-negative and often highly skewed



From EBI 2019 “RNA-Sequence-Analysis” school

Poisson distribution for RNA-seq read counts

Idea:

- Model the number of reads mapping to a gene in a sample as a **Poisson count**.

Model

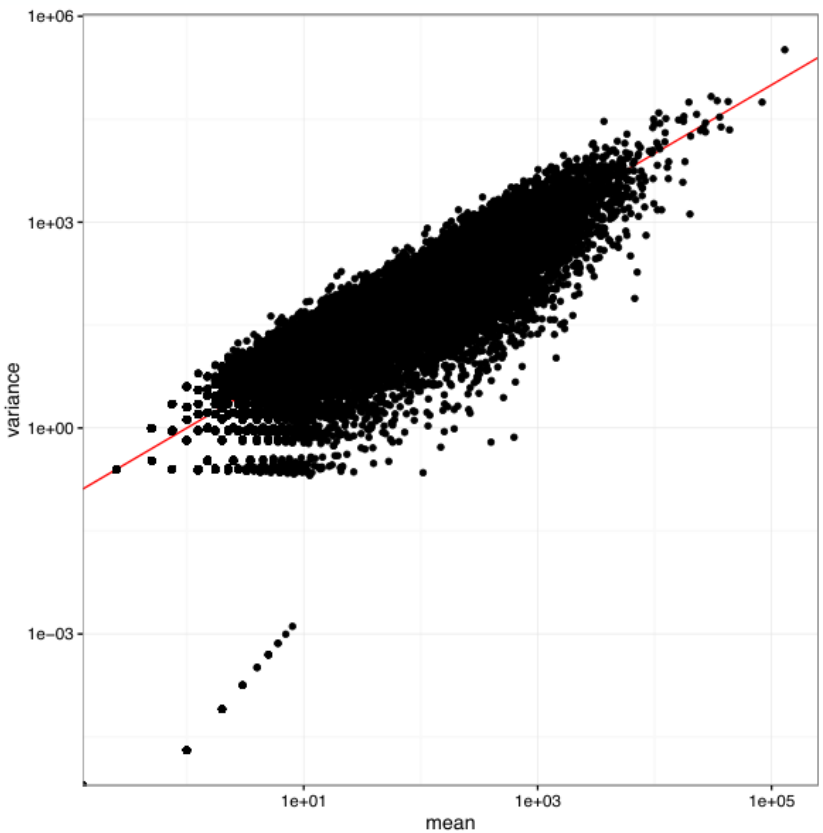
- Random variable:

$$Y_{gi} \sim Poisson(\lambda_{gi})$$

  where:
  - g= gene
  - i= sample
  - λ_{gi} =expected read count (depends on expression level, library size, etc.)
- Key Poisson property:

$$\mathbb{E}[Y_{gi}] = \lambda_{gi},$$
$$Var(Y_{gi}) = \lambda_{gi}$$

→**mean = variance**

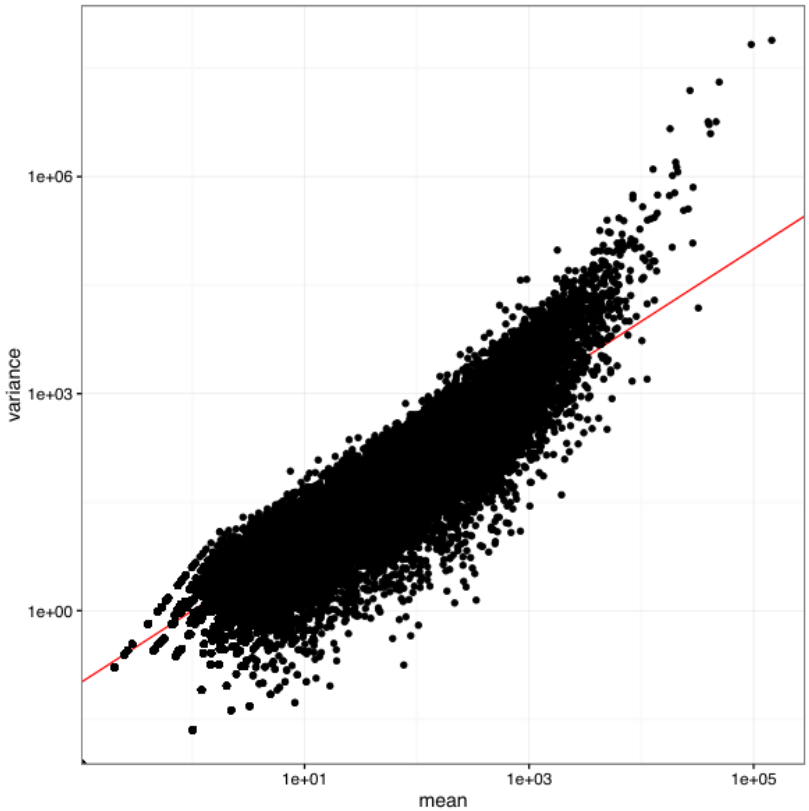


Why Poisson looked attractive for RNA-seq

- RNA-seq produces non-negative integer counts.
- Conceptual match: “number of events (reads) in a fixed interval (gene)”.

But… Poisson is usually *too simple*

- In real RNA-seq data, we see **overdispersion**:
  $$Var(Y_{g,i}) \gg \mathbb{E}[Y_{gi}]$$
- Extra variability comes from:
  - Biological differences between replicates
  - Technical variation (prep, sequencing)

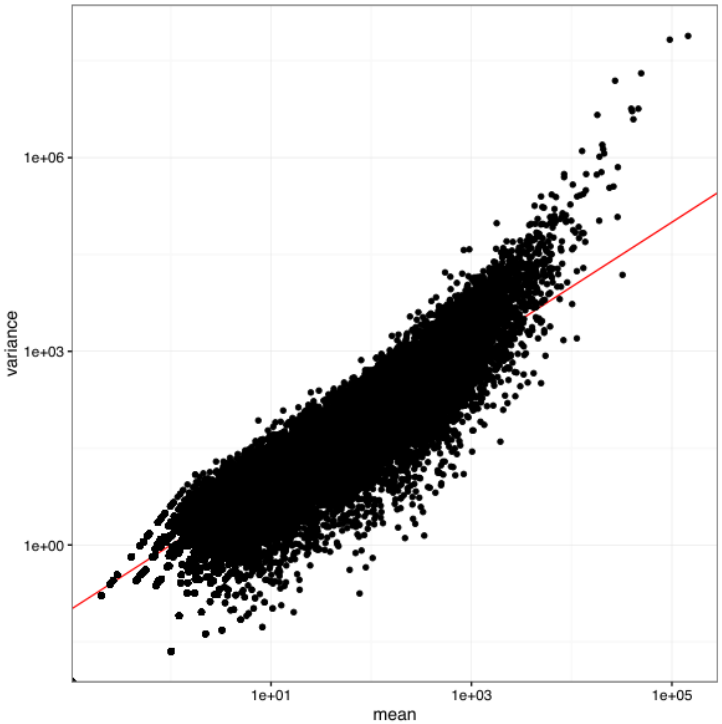


Modeling counts - the Negative Binomial distribution

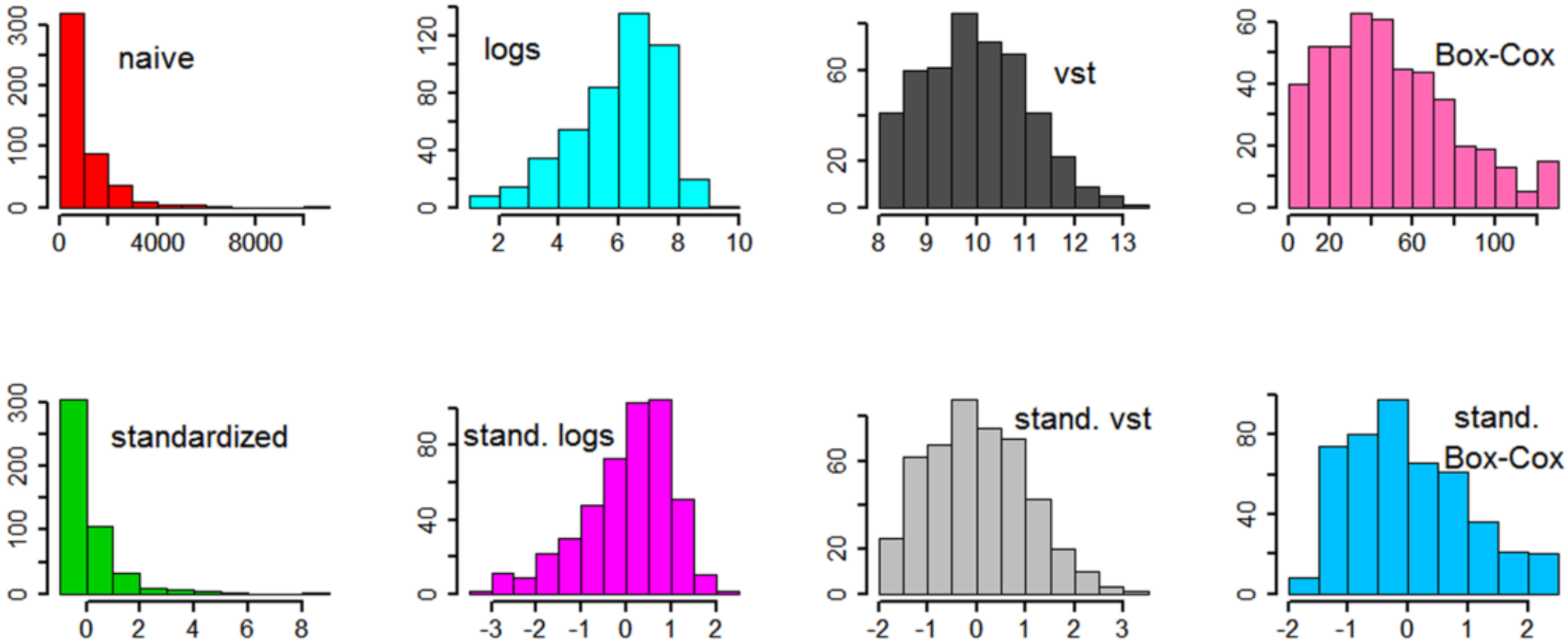
- Generalizes the Poisson distribution

$$Var(Y_{g,i}) = \mathbb{E}[Y_{gi}] + \theta \mathbb{E}[Y_{gi}]^2$$

- Θ = dispertion
- $\sqrt{\Theta}$ = "biological coefficient of variation"
- Allows mRNA proportions to vary across samples
- Captures variability across biological replicates better



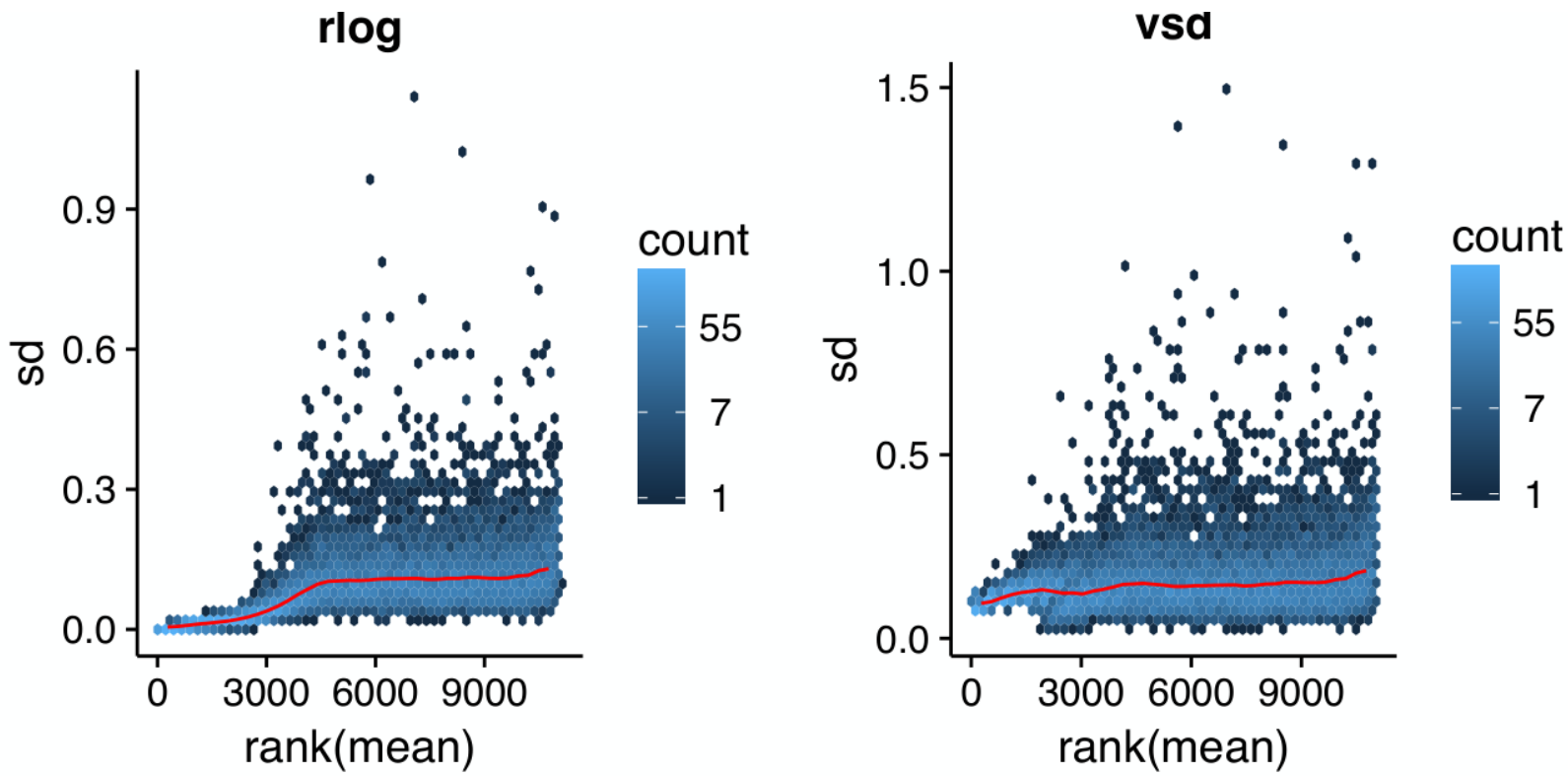
Many data transformations are possible



From EBI 2019 “RNA-Sequence-Analysis” school

Data transformations - DESeq2

- Two approaches: rlog, variance stabilizing transformation
- Aim: remove dependence of variance on mean after transformation



From EBI 2019 “RNA-Sequence-Analysis” school

DESeq2 vs edgeR: where does variance-stabilization happen?

In DESeq2

- DE is still done on **raw counts + NB model**, but:
  - For **visualization / clustering / distances**, DESeq2 offers:
    - **VST (variance stabilizing transformation)**
    - **rlog (regularized log)**
- These transforms aim to make:
  - variance ≈ constant across the range of means
  - so Euclidean distances / PCA behave nicely.
- Users explicitly **choose** VST or rlog

DESeq2 vs edgeR: where does variance-stabilization happen?

In edgeR for **DE testing**:

edgeR **always** works with:

- raw counts
- NB GLM:
- with dispersion ϕ_g estimated and shrunk.

The mean–variance relationship:

$$\mathrm{Var}(Y_{gi}) = \mu_{gi} + \phi_g \mu_{gi}^2$$

is **explicitly modeled**, so edgeR doesn't need a separate "variance-stabilizing transform choice" before DE.

So: **no rlog/vst step** — the NB model itself is the variance model.

What does edgeR use for plots / distances?

Instead of rlog/vst, edgeR typically uses:

- logCPM with a prior:
  - Used for PCA, heatmaps, etc.
  - The small prior.count stabilizes low counts
- plotMDS():
  - Works directly from the DGEList (its own data object)
  - it computes distances based on leading log-fold changes between samples (not just Euclidean on a transformed matrix)
  - This is another way of indirectly handling mean–variance differences without an explicit vst/rlog

Shrinkage dispersion estimation

DESeq2 models counts for gene *g* in sample *i* as Negative Binomial: $Y_{gi} \sim \mathrm{NB}(\mu_{gi}, \theta_g)$

μ_{gi} = the mean for gene *g* and sample *I*

θ_g = the dispersion

$$\mathrm{Var}(Y_{gi}) = \mu_{gi} + \theta_g \mu_{gi}^2$$

controls how much extra variability (overdispersion) the gene has beyond simple Poisson noise.

Shrinkage dispersion estimation

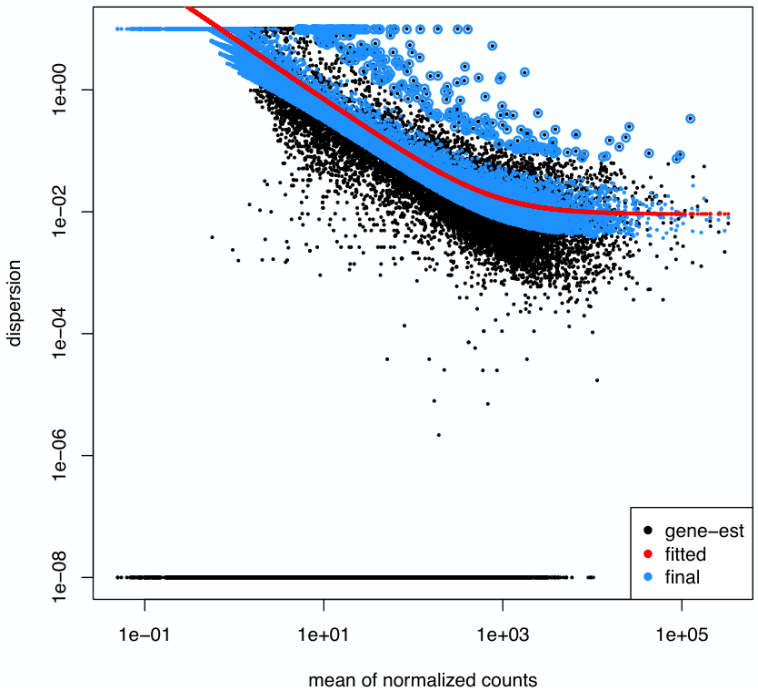
- In a typical RNA-seq experiment you might have:
  - 3–4 samples per condition
  - ~20,000 genes
- If you tried to estimate θ_g **independently for each gene** from those few replicates:
  - Lowly expressed genes → almost no information → dispersion estimates jump all over the place.
  - Even moderately expressed genes → estimates are very noisy with 3–4 observations.
  - You get many **extreme dispersions** (huge or almost zero) just by chance.
- Noisy dispersion means noisy variance :
  - *p*-values too small for some genes (false positives),
  - too large for others (loss of power),

Shrinkage dispersion estimation

DESeq2 uses **empirical Bayes shrinkage**:

1. Estimate a global trend of dispersion vs mean across all genes
   (genes with similar mean counts tend to have similar dispersions).
2. For each gene:
   - Compute its **raw** ("gene-wise") dispersion estimate from its small number of samples.
   - "Shrink" (pull) that raw estimate towards the **trend**.

- The final dispersion for gene *g* is a compromise between:
  - what the data for that gene say,
  - what all other genes collectively suggest for that expression level.
- This is a classic **bias–variance trade-off**:
  - You introduce **a little bias** (estimates move toward the trend),
  - but you **greatly reduce variance** (estimates are more stable).



Shrinkage dispersion estimation

Shrinkage dispersion estimation in DESeq2:

- **Stabilizes variance estimates** across genes, especially with few replicates.
- Prevents crazy outliers (e.g. genes with almost zero estimated dispersion or enormous dispersion just by chance).
- Gives **better controlled type I error** (FDR) and **more power**, because tests aren't based on wildly fluctuating gene-wise variances.
- Makes downstream results less sensitive to small quirks in a single gene's counts.

Shrinkage dispersion estimation: EdgeR

- edgeR models counts with a Negative Binomial too
- and it doesn't trust the per-gene dispersion ϕ_g when there are few replicates.

empirical Bayes shrinkage

Shrinkage dispersion estimation: EdgeR

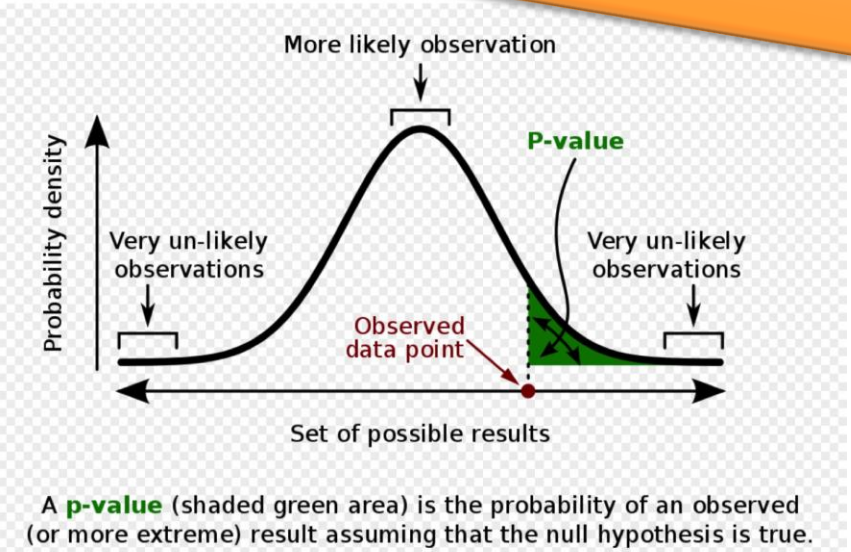
- Estimate a "global" dispersion
  - **Common dispersion**: single dispersion for all genes (very rough).
  - Or **trended dispersion**: smooth curve of dispersion vs abundance.
- **Estimate gene-wise ("tagwise") dispersions**
  - For each gene, it gets its own raw dispersion from the data.
- **Shrink tagwise dispersions toward the common/trended value**
  - The EB step pulls noisy gene-wise dispersions toward the overall trend.
  - Strength of shrinkage depends on how much information a gene has (counts, replicates) and on a prior parameter.

Result: you get stabilized tagwise dispersions, very similar in spirit to DESeq2's mean–dispersion trend + shrinkage but using empirical Bayes

P-value

- **Setting**
  - We want to test a **null hypothesis** H_0 (e.g. "gene X has the same mean expression in both conditions").
- **informal definition:** after computing a test statistic (e.g. t-statistic, Wald statistic), the **p-value** is:

Hence, if we get a p-value of 0.05, it means that there is a 5% chance of getting that extreme results **even in the absence of real signal**!



https://biocorecrg.github.io/CRG_Bioinformatics_for_Biologists/differential_gene_expression.html

The probability, *assuming* H_0 *is true*, of getting a result at least as extreme as the one we observed.

$$p = P\left(\begin{matrix}\textit{data as or more extreme than} \\ \textit{observed} \mid H_0 \textit{ true}\end{matrix}\right)$$

What does this mean for high-throughput studies?

- Assume that we perform 10,000 tests (one for each gene)…
- … and that there is no true signal at all in the data
- Then we would expect to get around 500 p-values below 0.05
- Relying solely on p-values would be misleading!

From EBI 2019 “RNA-Sequence-Analysis” school

Benjamini-Hochberg correction - controlling the FDR

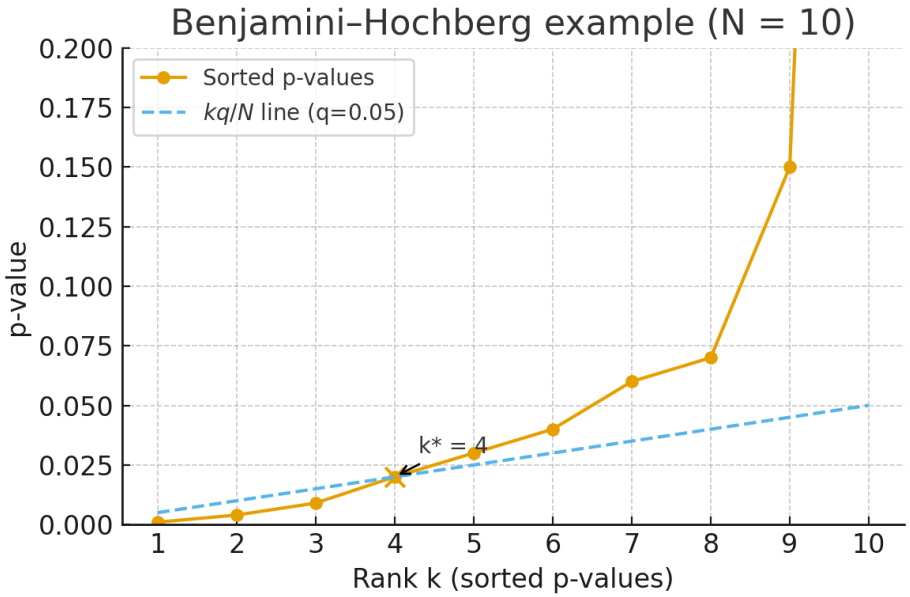
- Assume we are performing N tests
- Intuition:
  - for each threshold α, expected # false discoveries ≈ αN(if all null)
  - Compare this to the actual # discoveries at that threshold = $R(\alpha)$
  - Choose alpha so that $\alpha N / R_\alpha$ ≤ 0.05 (or another desired threshold)
  - **FDR =** expected proportion of false positives among all rejections
    $\mathrm{FDR} = E[V/R]$,where
    – V= # false discoveries,
    – R= # total discoveries (rejected H_0s).

BH algorithm

- Sort p-values: $p_{(1)} \leq \cdots \leq p_{(N)}$
- Choose desired FDR level q
- Find largest k such that

$$p_{(k)} \leq \frac{k}{N} q$$

- Reject all $H_{(1)}, \ldots, H_{(k)}$



BH-adjusted p-values

The **smallest FDR level** q at which a given test would be called significant

- In practice (R's p.adjust(method="BH")):

After sorting, compute:

$$q_{(i)} = \frac{N}{i} p_{(i)}$$

- Make them monotone non-decreasing from the bottom:

$$p_{(i)}^{adj} = \min_{j \geq i} q_{(j)}, \textit{capped at } 1$$

- Reorder them back to the original gene order.

If you choose FDR level $q = 0.05$, you reject all tests with $p_i^{adj} \leq 0.05$.

This is **equivalent** to the "find largest k" BH algorithm, but now you have an adjusted p-value **for each gene**.

Bonferroni vs BH

- **FWER (Bonferroni):** control probability of *any* false positive
  FDR (BH): control *proportion* of false positives among discoveries

Interpreting the FDR

- The FDR is a measure for a set of genes
- In a set of genes with FDR = 0.05, approximately 5% can be expected to be
- false discoveries
- However, we don’t know which ones! It could be the most significant!